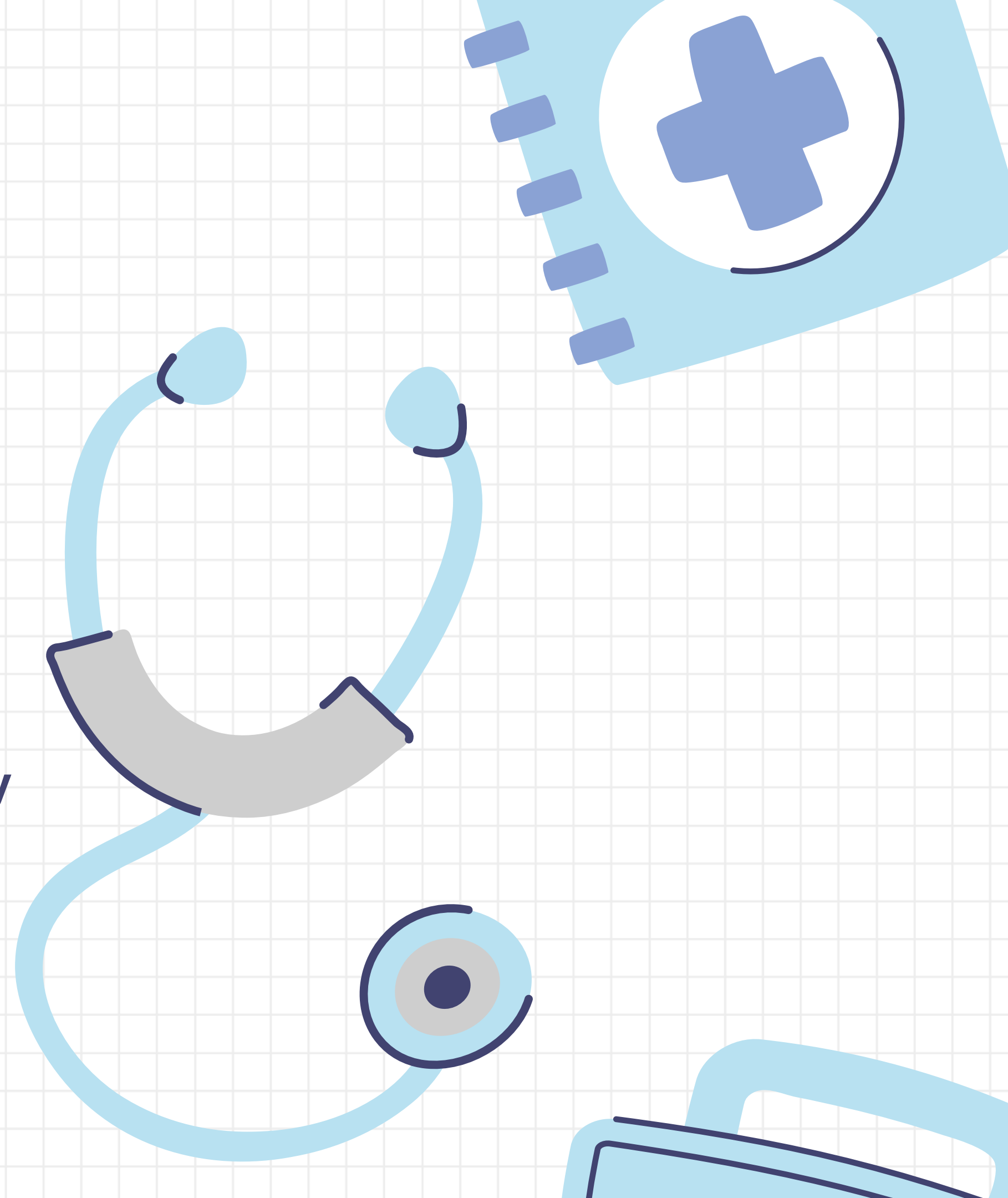


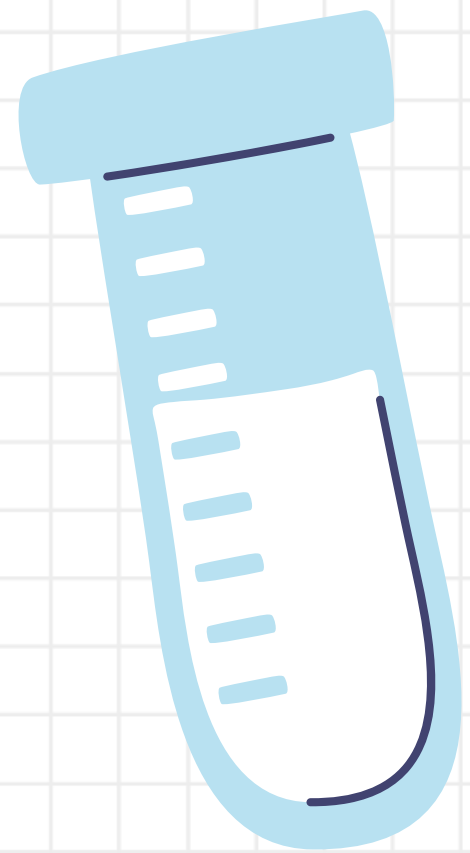
**Butemwe Nkinke & Keila
Ngandu**

**Supervisor: Mrs Monde
Kabemba**

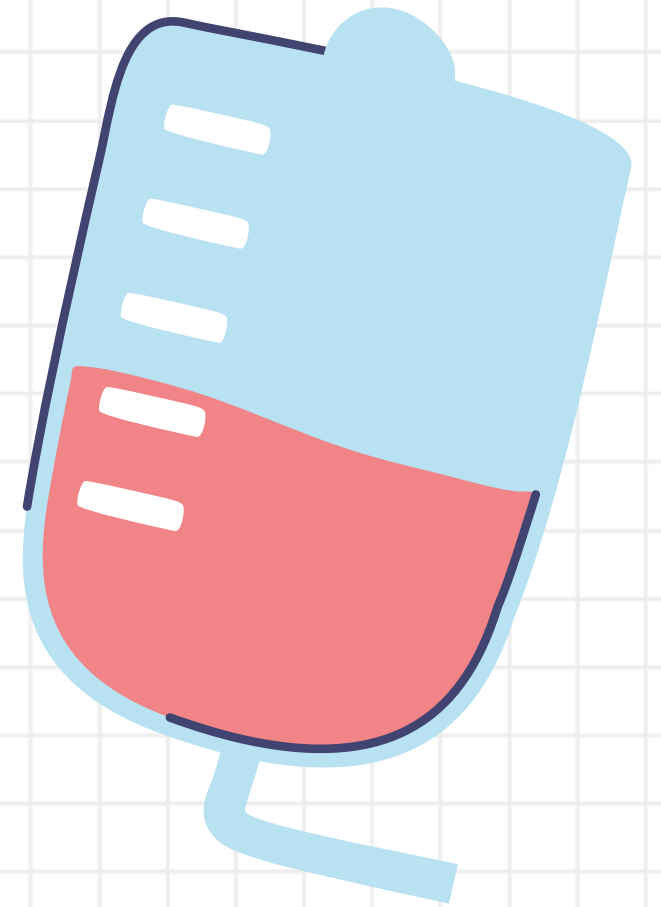
MediVault

**A Role-Based Patient Portal for the University
of Zambia Clinic
2026**





The Problem



No Medical History

Doctors do not always have access to a patient's medical history at the point of care. As a result, treatment decisions may be made without full knowledge of existing conditions, medications, or allergies.

Physical ID Required

Students are required to physically present their university ID cards to confirm their enrolment before receiving care. This reliance on physical documents can be problematic when cards are lost, forgotten, or simply unavailable.

Emergency Consent

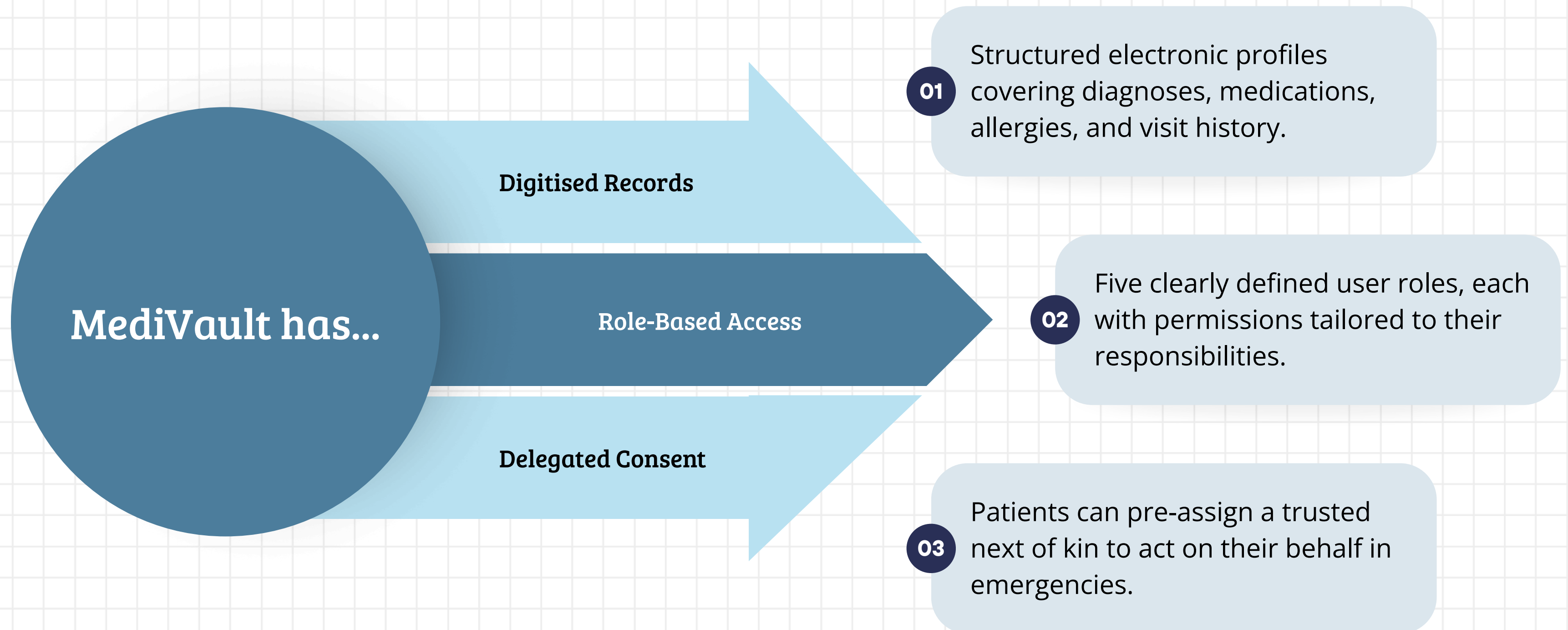
Lack of a formal process for accessing patient information or obtaining consent when a patient is physically unable to give consent. This becomes especially critical in emergency situations where timely decisions are required.

Accountability Trail

There is also no structured method to monitor who accesses or modifies patient records, which raises concerns about accountability and the integrity of the data.

What is MediVault?

MediVault is a secure, web-based patient portal built specifically for the UNZA Clinic. It digitises student medical records, replaces physical ID checks with student number verification, and introduces a clear role-based access structure alongside a delegated consent mechanism.



Project Aim

The goal is to design and build MediVault, a secure, role based web portal for the UNZA clinic. The system will digitise student medical records, remove the need for physical ID checks by allowing verification through student numbers, and introduce clear access controls along with delegated consent features. Together, these improvements aim to make patient care safer, more efficient, and more reliable.

Objectives

- 01 Design and implement a role based access control system, with clear permissions assigned to patients, receptionists, doctors, and administrators.
- 02 Develop a student verification feature that allows receptionists to confirm eligibility using a student number, removing the need for physical ID cards.
- 03 Create a structured electronic medical profile for each patient, including diagnoses, medications, allergies, and visit history, so authorised clinical staff can easily access it during care.
- 04 Add a next of kin consent feature that lets patients pre-authorise a trusted person to make decisions on their behalf if they become incapacitated.
- 05 The system will include a simple logging mechanism that records when patient records are accessed or updated, along with the user responsible. This will provide a basic level of accountability and transparency without introducing unnecessary complexity

User Roles & Permissions

Patient

01

- View own medical profile
- Assign next of kin
- Review visit history

Receptionist

02

- Verify student identity via student number
- Register walk-ins
- Manage appointments

Doctor

03

- Access full medical history
- Add diagnoses and notes
- Prescribe medications

Administrator

04

- Manage user accounts
- Monitor access logs
- Configure system settings

Next of Kin

05

- Activated on patient delegation
- Provide consent for treatment
- View basic patient details

Key Features

Student Number Verification

Replaces physical ID card checks. Receptionists can confirm eligibility by looking up a student's registered number directly in the system.

Access Logging

A simple but effective audit trail records when records are viewed or modified and which user was responsible, providing basic accountability within the system.

Electronic Medical Profiles

Each patient has a structured profile with diagnoses, current medications, known allergies, and a full visit history accessible to authorised clinical staff.

Next of Kin Consent Mechanism

Patients can pre-assign a trusted individual during registration. If the patient becomes incapacitated, the system enables the next of kin to provide consent for treatment.

Proposed Methodology

Development Approach

The project will follow an Agile inspired approach, where the system is built in small, manageable parts rather than all at once. A modular, feature by feature approach will be used, meaning each feature (e.g., viewing medical records) will include its interface, backend logic, and database interaction.

Version control will be handled using GitHub, allowing team members to work on different parts of the system simultaneously.

System Architecture / Workflow

The system will use a full stack architecture based on Next.js.

Users interact with the frontend, which sends requests to backend API routes. The backend processes the request, applies role based access control, and retrieves or updates data in the PostgreSQL database. The response is then returned and displayed to the user.

The system will be organised into modules such as authentication, medical records, and next of kin consent.

Data Collection

No real patient data will be used. Instead, fake data will be created to simulate students, medical records, and user roles for testing and demonstration purposes.

Technology Stack

Languages

HTML · CSS · JavaScript

Framework

Next.js (Frontend + API routes)

Authentication

JSON Web Tokens (JWT)

Development Tools

Visual Studio Code · pgAdmin · GitHub

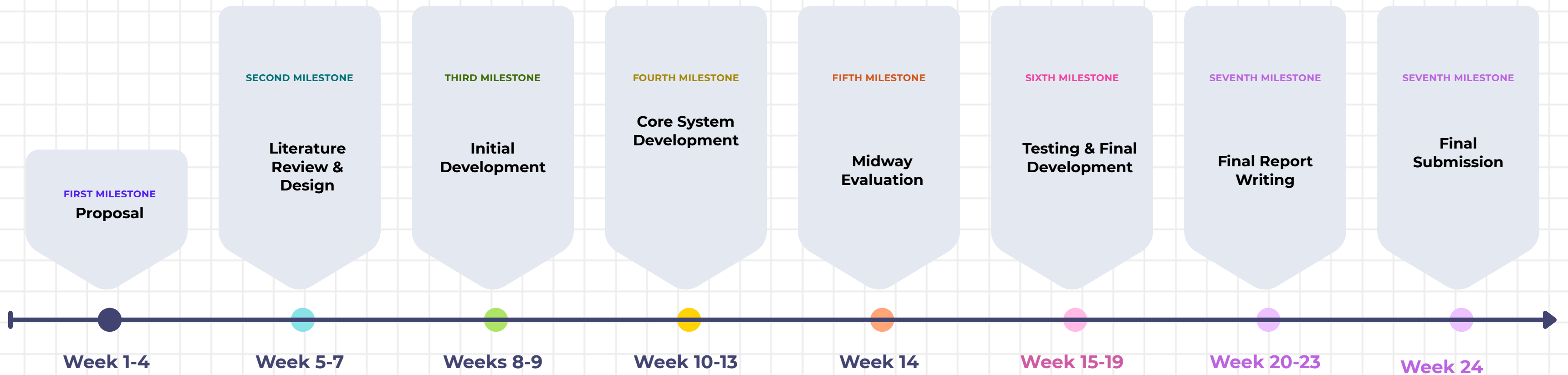
Version Control

GitHub (Collaborative sprint workflow)

Databases

PostgreSQL

Timeline



Expected Outcomes

A fully functional web-based patient portal deployed for the UNZA clinic environment.

Role-based access control for five user types, with clearly defined permissions at every level.

A student identity verification feature that removes the need for physical ID cards at the clini

Structured electronic medical profiles enabling efficient access to patient information during

A next of kin and delegated consent workflow to cover emergency situations.

Secure data storage with access logging for accountability and transparency.

Full technical documentation including system architecture diagrams, API docs, and per-role user guide.

Risk Analysis

Risk: Access to real student data for testing

Mitigation: Use synthetically generated test data throughout development; request anonymised sample data only for final validation if approved.

Risk: Scope creep beyond clinic context

Mitigation: Adhere strictly to the defined scope; document any supervisor approved scope changes formally.

Risk: Security vulnerabilities in the portal

Mitigation: Follow basic security best practices and perform simple security testing during development.

Risk: Team member unavailability

Mitigation: Maintain shared code repository on GitHub; document all progress in regular sprint notes.

Risk: Technology compatibility issues

Mitigation: Use widely supported, LTS versions of all frameworks; validate choices early in the setup phase.

Risk: Insufficient user testing participants

Mitigation: Engage fellow students early to recruit testers; conduct testing iteratively across multiple sprints.

Risk: Authentication or session handling issues (JWT)

Mitigation: Test authentication flows thoroughly and validate token handling during development.

Thank you!

